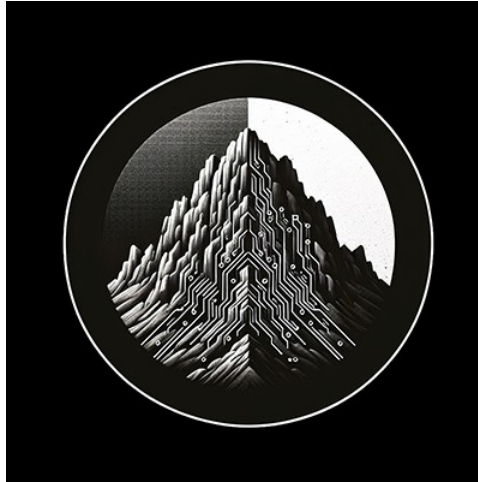




RTL Design Sherpa

RTL AMBA AXI4-Lite Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXIL4 (AXI4-Lite) Modules.....	9
1.1 Overview.....	9
1.1.1 Key Features.....	9
1.2 Module Categories.....	10
1.2.1 Core Master/Slave Modules.....	10
1.2.2 Monitor Modules (Verification).....	10
1.2.3 Clock-Gated Variants.....	11
1.3 AXI4-Lite Protocol Overview.....	11
1.3.1 Simplified vs Full AXI4.....	11
1.3.2 Channel Architecture.....	12
1.3.3 Key Signals (Simplified).....	12
1.4 Quick Start.....	12
1.4.1 Basic Read Master.....	12
1.4.2 Basic Write Slave.....	13
1.4.3 Monitor Integration.....	13
1.5 Testing.....	14
1.6 Terminology.....	15
1.7 Known Limitations.....	15
1.8 Design Patterns.....	16
1.8.1 Pattern 1: Buffered Master/Slave.....	16
1.8.2 Pattern 2: Monitor Integration.....	16
1.8.3 Pattern 3: Clock Gating.....	16

1.9 Performance Characteristics.....	16
1.9.1 Throughput.....	16
1.9.2 Latency.....	17
1.9.3 Resource Usage.....	17
1.10 Related Documentation.....	17
1.10.1 Protocol Specifications.....	17
1.10.2 RTL Design Sherpa Documentation.....	18
1.10.3 Configuration and Integration.....	18
1.10.4 Source Code.....	18
1.11 Design Notes.....	18
1.11.1 When to Use AXI4-Lite.....	18
1.11.2 Buffer Depth Guidelines.....	18
1.12 Navigation.....	19
2 AXI4 Clock-Gated Variants Guide.....	19
2.1 Overview.....	19
2.1.1 Available Clock-Gated Modules.....	19
2.1.2 Key Features.....	20
2.2 Additional Parameters (All _cg Modules).....	20
2.3 Additional Ports (All _cg Modules).....	20
2.3.1 Clock Gating Configuration.....	20
2.3.2 Clock Gating Status.....	21
2.4 Usage Example.....	21
2.5 Clock Gating Behavior.....	22
2.6 Configuration Guidelines.....	22

2.6.1 Idle Count Selection.....	22
2.6.2 When to Use Clock Gating.....	23
2.7 Power Savings Estimates.....	23
2.7.1 Typical Savings (AXI4-Lite Register Access).....	23
2.8 Complete Documentation.....	24
2.9 Related Documentation.....	24
2.9.1 Base Modules.....	24
2.9.2 Architecture.....	24
2.10 Navigation.....	24
3 AXI4 Master Read.....	24
3.1 Overview.....	25
3.1.1 Key Features.....	25
3.2 Parameters.....	25
3.2.1 Derived Parameters.....	25
3.3 Port Groups.....	26
3.3.1 Clock and Reset.....	26
3.3.2 Frontend AXI4-Lite Read Interface (Input Side).....	26
3.3.3 Backend AXI4-Lite Master Interface (Output Side).....	26
3.3.4 Status Outputs.....	27
3.4 Signal Flow.....	28
3.4.1 Read Transaction Sequence.....	28
3.5 Usage Example.....	28
3.5.1 Basic Integration.....	28
3.5.2 Register Read Example (Testbench Stimulus).....	29

3.6 Design Notes.....	30
3.6.1 AXI4-Lite Protocol Constraints.....	30
3.6.2 Buffer Depth Selection.....	30
3.6.3 Busy Signal.....	30
3.6.4 PROT Encoding.....	31
3.7 Performance Characteristics.....	31
3.7.1 Latency.....	31
3.7.2 Throughput.....	31
3.7.3 Resource Usage.....	31
3.8 Related Modules.....	32
3.8.1 Core Modules.....	32
3.8.2 Monitor Modules.....	32
3.8.3 Clock-Gated Variants.....	32
3.8.4 Supporting Infrastructure.....	32
3.9 References.....	32
3.9.1 Specifications.....	32
3.9.2 Documentation.....	32
3.9.3 Source Code.....	32
3.10 Navigation.....	33
4 AXI4 Master Read Interface (Clock-Gated).....	33
4.1 Quick Reference.....	33
4.2 Summary.....	33
4.3 Common Parameters.....	34
4.3.1 Additional Ports.....	34

4.4 Quick Usage.....	34
4.5 Documentation.....	35
4.6 Navigation.....	35
5 AXI4 Master Write.....	36
5.1 Overview.....	36
5.1.1 Key Features.....	36
5.2 Parameters.....	36
5.2.1 Derived Parameters.....	37
5.3 Port Groups.....	37
5.3.1 Clock and Reset.....	37
5.3.2 Frontend AXI4-Lite Write Interface (Input Side).....	37
5.3.3 Backend AXI4-Lite Master Interface (Output Side).....	38
5.3.4 Status Outputs.....	38
5.4 Signal Flow.....	40
5.4.1 Write Transaction Sequence.....	40
5.5 Usage Example.....	40
5.5.1 Basic Integration.....	40
5.5.2 Register Write Example (Testbench Stimulus).....	41
5.5.3 Partial Write Example (Byte Strobes).....	42
5.6 Design Notes.....	42
5.6.1 AXI4-Lite Protocol Constraints.....	42
5.6.2 Buffer Depth Selection.....	43
5.6.3 Write Strobe (WSTRB) Usage.....	43
5.6.4 Busy Signal.....	43

5.7 Performance Characteristics.....	44
5.7.1 Latency.....	44
5.7.2 Throughput.....	44
5.7.3 Resource Usage.....	45
5.8 Related Modules.....	45
5.8.1 Core Modules.....	45
5.8.2 Monitor Modules.....	45
5.8.3 Clock-Gated Variants.....	45
5.8.4 Supporting Infrastructure.....	45
5.9 References.....	45
5.9.1 Specifications.....	45
5.9.2 Documentation.....	46
5.9.3 Source Code.....	46
5.10 Navigation.....	46
6 AXIL4 Master Write Interface (Clock-Gated).....	46
6.1 Quick Reference.....	46
6.2 Summary.....	47
6.3 Common Parameters.....	47
6.3.1 Additional Ports.....	47
6.4 Quick Usage.....	48
6.5 Documentation.....	48
6.6 Navigation.....	49
7 AXIL4 Slave Read.....	49
7.1 Overview.....	49

7.1.1 Key Features.....	49
7.2 Parameters.....	49
7.3 Port Groups.....	50
7.3.1 Slave Interface (Input Side from Interconnect).....	50
7.3.2 Backend Interface (Output Side to Memory/Logic).....	50
7.3.3 Status Outputs.....	50
7.4 Usage Example.....	50
7.5 Design Notes.....	51
7.6 Related Modules.....	51
7.7 Navigation.....	52
8 AXIL4 Slave Read Interface (Clock-Gated).....	52
8.1 Quick Reference.....	52
8.2 Summary.....	52
8.3 Common Parameters.....	53
8.3.1 Additional Ports.....	53
8.4 Quick Usage.....	54
8.5 Documentation.....	54
8.6 Navigation.....	55
9 AXIL4 Slave Write.....	55
9.1 Overview.....	55
9.1.1 Key Features.....	55
9.2 Parameters.....	55
9.3 Port Groups.....	56
9.3.1 Slave Interface (Input Side from Interconnect).....	56

9.3.2 Backend Interface (Output Side to Memory/Logic).....	56
9.3.3 Status Outputs.....	57
9.4 Usage Example.....	57
9.5 Design Notes.....	58
9.6 Related Modules.....	58
9.7 Navigation.....	58
10 AXIL4 Slave Write Interface (Clock-Gated).....	59
10.1 Quick Reference.....	59
10.2 Summary.....	59
10.3 Common Parameters.....	59
10.3.1 Additional Ports.....	60
10.4 Quick Usage.....	60
10.5 Documentation.....	61
10.6 Navigation.....	61

1 AXIL4 (AXI4-Lite) Modules

Location: rtl/amba/axil4/ **Test Location:** val/amba/ **Status:** ✓ Production Ready

1.1 Overview

The AXIL4 subsystem provides a complete implementation of the ARM AMBA AXI4-Lite protocol—a simplified subset of AXI4 designed for control register access with reduced signal count and single-beat transactions.

1.1.1 Key Features

- ✓ **AXI4-Lite Protocol:** Simplified subset (no burst, no ID, no QoS)
- ✓ **Single-Beat Transactions:** Always 1 transfer per transaction
- ✓ **Reduced Signals:** 40-50% fewer signals vs full AXI4

- ✓ **Buffered Interfaces:** Elastic buffering via `gaxi_skid_buffer`
- ✓ **Monitoring Infrastructure:** Integrated transaction monitoring
- ✓ **Clock Gating Variants:** Power-optimized versions
- ✓ **Typical Use:** Control registers, CSRs, peripheral access

1.2 Module Categories

1.2.1 Core Master/Slave Modules

Module	Description	Documentation	Status
axil4_master_rd	AXIL4 read master with buffered channels	axil4_master_rd.md	✓ Documented
axil4_master_wr	AXIL4 write master with buffered channels	axil4_master_wr.md	✓ Documented
axil4_slave_rd	AXIL4 read slave with buffered channels	axil4_slave_rd.md	✓ Documented
axil4_slave_wr	AXIL4 write slave with buffered channels	axil4_slave_wr.md	✓ Documented

1.2.2 Monitor Modules (Verification)

Monitor wrappers live in `rtl/amba/monitor/` (not `rtl/amba/axil4/`). They are dedicated AXIL4 wrappers — there is no IS_AXI-style parameter overload of the AXI4 wrappers — and they share `axi_monitor_base` and the 128-bit packet format with the AXI4 wrappers.

Module	Description	Documentation	Status
axil4_master_rd_mon	Read master with integrated monitoring	axil4_master_rd_mon.md	✓ Documented
axil4_master_wr_mon	Write master with integrated monitoring	axil4_master_wr_mon.md	✓ Documented
axil4_slave_rd_mon	Read slave with integrated monitoring	axil4_slave_rd_mon.md	✓ Documented
axil4_slave_wr_mon	Write slave with integrated monitoring	axil4_slave_wr_mon.md	✓ Documented

1.2.3 Clock-Gated Variants

All clock-gated variants documented in: [axil4_clock_gating_guide.md](#)

Module	Base Module	Status
axil4_master_rd_cg	axil4_master_rd	✓ Documented
axil4_master_wr_cg	axil4_master_wr	✓ Documented
axil4_slave_rd_cg	axil4_slave_rd	✓ Documented
axil4_slave_wr_cg	axil4_slave_wr	✓ Documented
axil4_master_rd_mon_cg	axil4_master_rd_mon	✓ Documented
axil4_master_wr_mon_cg	axil4_master_wr_mon	✓ Documented
axil4_slave_rd_mon_cg	axil4_slave_rd_mon	✓ Documented
axil4_slave_wr_mon_cg	axil4_slave_wr_mon	✓ Documented

1.3 AXI4-Lite Protocol Overview

1.3.1 Simplified vs Full AXI4

Feature	AXI4	AXI4-Lite
Burst Support	Yes (1-256 beats)	No (always 1 beat)
ID Signals	ARID/AWID/RID/BID	None
Out-of-Order	Supported via ID	Not supported
Data Width	8-1024 bits	32 or 64 bits
QoS/Region	Supported	Not supported
Lock/Cache	Full support	None (no AxLOCK, no AxCACHE)
Signal Count	~40-50 signals	~20-25 signals
Typical Use	High-bandwidth data	Control registers

1.3.2 Channel Architecture

AXI4-Lite defines five independent channels. Any one transaction uses only a subset of them: a write uses three (AW, W, B) and a read uses two (AR, R). The read and write channel sets are fully independent and can be in flight concurrently.

Write Transaction (3 channels): 1. **AW (Write Address)** - Address and protection 2. **W (Write Data)** - Data and byte strobes 3. **B (Write Response)** - Completion status

Read Transaction (2 channels): 1. **AR (Read Address)** - Address and protection 2. **R (Read Data)** - Data and response

1.3.3 Key Signals (Simplified)

Address Channels (AR/AW): - ARADDR/AWADDR - Byte address - ARPROT/AWPROT - Protection attributes (3 bits) - ARVALID/AWVALID, ARREADY/AWREADY - Handshake

Data Channels (R/W): - RDATA/WDATA - Transfer data (32 or 64 bits) - RRESP/BRESP - Response (OKAY, SLVERR, DECERR) - WSTRB - Write strobes (byte lane enables) - RVALID/WVALID, RREADY/WREADY - Handshake

1.4 Quick Start

1.4.1 Basic Read Master

```
axil4_master_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),      // 2 entries
    .SKID_DEPTH_R(4)        // 4 entries
) u_rd_master (
    .aclk                (axi_clk),
    .aresetn              (axi_resetn),

    // Frontend interface (from CPU)
    .fub_araddr          (cpu_araddr),
    .fub_arprot           (cpu_arprot),
    .fub_arvalid          (cpu_arvalid),
    .fub_arready          (cpu_arready),

    .fub_rdata            (cpu_rdata),
    .fub_rresp            (cpu_rresp),
    .fub_rvalid           (cpu_rvalid),
    .fub_rready           (cpu_rready),

    // Master interface (to interconnect)
```

```

        .m_axil_araddr (periph_araddr),
        // ... rest of AR/R signals

    .busy          (rd_busy)
);

```

1.4.2 Basic Write Slave

```

axil4_slave_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_wr_slave (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_awaddr (interconnect_awaddr),
    // ... rest of AW/W/B signals

    // Backend interface (to register block)
    .fub_awaddr    (regfile_awaddr),
    // ... rest of AW/W/B signals

    .busy          (wr_busy)
);

```

1.4.3 Monitor Integration

```

axil4_master_rd_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(1),
    .AGENT_ID(10),
    .MAX_TRANSACTIONS(8), // Reduced for AXIL (axi4*_mon defaults
to 16)
    .ENABLE_FILTERING(1)
) u_rd_mon (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // AXIL interfaces
    .fub_axil_araddr (cpu_araddr),
    // ... rest of AR/R signals

    // Monitor configuration

```

```

        .cfg_monitor_enable      (1'b1),
        .cfg_error_enable       (1'b1),
        .cfg_timeout_enable     (1'b1),
        .cfg_perf_enable        (1'b0), // Avoid congestion
        .cfg_timeout_cycles     (16'd1000),

        // Filtering masks - see note below on polarity
        .cfg_axi_pkt_mask       (16'b1111_1111_1111_0110), // Keep Error
+ Timeout
        // ... rest of masks

        // Monitor bus output
        .monbus_valid           (mon_valid),
        .monbus_ready           (mon_ready),
        .monbus_packet          (mon_packet),
        .monbus_timestamp       (mon_timestamp),

        // Status
        .busy                   (rd_busy),
        .active_transactions    (rd_active),
        .error_count            (rd_errors)
    );

```

cfg_axi_pkt_mask polarity and indexing. The mask is 16 bits because it is indexed by the 4-bit packet_type field of the monitor packet — one mask bit per packet type — not by the 128-bit packet width. A **set bit drops** that packet type (pkt_drop = cfg_axi_pkt_mask[pkt_type] in axi_monitor_filtered). So 16'b1111_1111_1111_0110 clears bit 0 (PktTypeError) and bit 3 (PktTypeTimeout) to let those through and drops everything else. See the [Monitor Packet Specification](#) for the packet-type enum.

1.5 Testing

All AXIL4 modules are verified using CocoTB-based testbenches:

Run all AXIL4 tests

```
pytest val/amba/test_axil4*.py -v
```

Run specific module tests

```
pytest val/amba/test_axil4_master_rd.py -v
```

```
pytest val/amba/test_axil4_slave_wr.py -v
```

```
pytest val/amba/test_axil4_master_rd_mon.py -v
```

Run with waveforms

```
pytest val/amba/test_axil4_master_rd.py --vcd=waves.vcd -v
```

1.6 Terminology

Term	Meaning
FUB	Functional Unit Block — the design-internal logic on the far side of the module from the external AXI4-Lite bus. FUB-side ports are prefixed <code>fub_</code> .
Skid buffer	A small elastic FIFO (<code>gaxi_skid_buffer</code>) placed in a valid/ready channel. It registers the handshake so neither side sees a combinational path to the other, at the cost of one clock of latency.
CG	Clock Gating. The <code>_cg</code> module suffix denotes the clock-gated variant.
MonBus	The 128-bit monitor packet bus (plus 64-bit side-band timestamp) emitted by the <code>_mon</code> wrappers.

1.7 Known Limitations

- **No clock domain crossing.** All modules are fully synchronous to `ac1k`. Cross domains with `gaxi_skid_buffer_async` or a CDC FIFO outside these modules.
- **No address decode or protection enforcement.** The modules are transport only: `AxPROT` is carried through untouched and never checked. Address decode belongs to the interconnect or the register block.
- **No error injection or response rewriting.** `RRESP` / `BRESP` pass through from the backend unmodified.
- **No write-data reordering or AW/W matching.** The `AW` and `W` channels are buffered independently; nothing in these modules pairs an address with its data. A backend that requires `AW` before `W` must enforce that itself.
- **Unaligned and narrow accesses are the backend's problem.** AXI4-Lite has no `AxSIZE`; partial-word access is expressed only through `WSTRB`, which the backend must honor.

- **gaxi_skid_buffer** constrains **SKID_DEPTH_*** to {2, 4, 6, 8}. Other values are not supported.
 - **Clock gating is ASIC-oriented.** `clock_gate_ctrl` instantiates an ICG cell; on FPGA this does not map to true clock gating. There is no scan/test bypass port — set `cfg_cg_enable = 0` to disable gating.
-

1.8 Design Patterns

1.8.1 Pattern 1: Buffered Master/Slave

All core modules use `gaxi_skid_buffer`: - Decouples frontend and backend timing - Configurable depth per channel - 1-cycle latency overhead - Full backpressure handling

1.8.2 Pattern 2: Monitor Integration

Monitor modules combine functional core with verification: - Uses shared **axi_monitor_filtered** (`rtl/amba/monitor/`) - 3-level filtering hierarchy - 128-bit monitor bus + 64-bit side-band timestamp - Simplified for AXIL (`MAX_TRANSACTIONS=8`, vs 16 for the AXI4 wrappers)

1.8.3 Pattern 3: Clock Gating

Clock-gated variants (`*_cg`) add power management: - Dynamic gating based on channel valid activity plus the base module's busy - Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. AXI4 is single-stage, so the first usable gated-clock edge arrives 2 clocks after activity asserts — see the [Clock-Gated Variants Guide](#) - Better power savings than AXI4 (bursty register access)

1.9 Performance Characteristics

1.9.1 Throughput

Configuration	Rate	Notes
Single-beat read/write	1 txn/cycle	When buffers not stalled
Typical peripheral	1 txn/N cycles	N = backend response time

1.9.2 Latency

Latencies below are the module's own buffering overhead and **exclude** slave response time. Each channel traversed costs one clock in its skid buffer.

Path	Cycles	Notes
AR → R (single)	Slave latency + 2	1 clock in the AR buffer, 1 in the R buffer
AW+W → B (single)	Slave latency + 3	AW and W buffers, then the B buffer
Buffer overhead	+1 per channel	Skid buffer latency

The write figure assumes the slave accepts AW and W in the same cycle. If the slave's AWREADY and WREADY assert in different cycles, the write costs the later of the two handshakes instead.

1.9.3 Resource Usage

The figures below are **order-of-magnitude estimates only**. They are not measured synthesis results: no target device, tool version, or optimization setting is attached to them. Treat them as relative sizing between the variants, and run synthesis for your own device before budgeting area.

Module Type	LUTs	FFs	BRAM	Notes
Master/ Slave (32-bit)	~200	~150	0	Per direction (rd/wr)
Monitor (+mon)	+800	+600	0	Additional overhead
Clock-gated (+cg)	+50	+30	0	Clock gating logic

vs AXI4: roughly 40-50% smaller, on the same estimate basis (no ID tracking, no burst counters, simpler protocol). This is an estimate, not a measured comparison against the AXI4 modules.

1.10 Related Documentation

1.10.1 Protocol Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification
- Chapter 4: AXI4-Lite

1.10.2 RTL Design Sherpa Documentation

- [RTLAmba Overview](#) - Complete AMBA subsystem architecture
- [AXI4 Modules](#) - Full AXI4 protocol (comparison)
- [APB Modules](#) - Even simpler peripheral bus
- [GAXI Modules](#) - Generic AXI utilities (buffers, FIFOs)

1.10.3 Configuration and Integration

- [AXI Monitor Configuration Guide](#) - Monitor setup strategies
- [Monitor Packet Specification](#) - 128-bit packet plus 64-bit side-band timestamp

1.10.4 Source Code

- RTL: rtl/amba/axil4/
 - Monitor wrappers: rtl/amba/monitor/
 - Tests: val/amba/test_axil4*.py
 - Framework: bin/TBClasses/components/axil4/
 - Shared Infrastructure: rtl/amba/monitor/ (monitor base and filter),
rtl/amba/gaxi/ (gaxi_skid_buffer)
-

1.11 Design Notes

1.11.1 When to Use AXI4-Lite

✓ **Use AXI4-Lite For:** - Control and status registers (CSRs) - Peripheral configuration interfaces - Low-bandwidth control paths - Resource-constrained designs - Simpler protocol requirements

✗ **Use Full AXI4 For:** - High-bandwidth data transfers - Burst transactions required - Out-of-order support needed - DMA engines, memory controllers - Streaming data

1.11.2 Buffer Depth Guidelines

The SKID_DEPTH_* parameters are passed straight through to the DEPTH parameter of gaxi_skid_buffer, which is the **literal entry count**, not an exponent. SKID_DEPTH_AR = 2 means a 2-entry buffer. gaxi_skid_buffer constrains DEPTH to one of {2, 4, 6, 8}.

Address Channels (AR/AW): - Default: 2 entries - sufficient for most register access - Increase for high-latency address decode

Data Channels (R/W): - Default: 4 entries for R, 2 entries for W - R channel typically deeper (accommodate backend latency) - W channel can be shallower (single-beat)

Response Channel (B): - Default: 2 entries - responses are single-beat - Rarely needs adjustment

Last Updated: 2026-07-19

1.12 Navigation

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

2 AXIL4 Clock-Gated Variants Guide

Location: rtl/amba/axil4/*_cg.sv **Status:** ✓ Production Ready

2.1 Overview

All AXIL4 modules have clock-gated (_cg) variants that add power management through dynamic clock gating. Identical architecture to [AXI4 Clock Gating](#) but optimized for AXI4-Lite's simpler protocol.

2.1.1 Available Clock-Gated Modules

Module	Base Module	Description
axil4_master_rd_cg	axil4_master_rd	Clock-gated read master
axil4_master_wr_cg	axil4_master_wr	Clock-gated write master
axil4_slave_rd_cg	axil4_slave_rd	Clock-gated read slave
axil4_slave_wr_cg	axil4_slave_wr	Clock-gated write slave
axil4_master_rd_mon_cg	axil4_master_rd_mon	Clock-gated read master + monitor
axil4_master_wr_mon_cg	axil4_master_wr_mon	Clock-gated write master + monitor
axil4_slave_rd_mon_cg	axil4_slave_rd_mon	Clock-gated read slave + monitor

Module	Base Module	Description
axil4_slave_wr_mon_cg	axil4_slave_wr_mon	Clock-gated write slave + monitor

The four *_mon_cg modules live in rtl/amba/monitor/; the four plain *_cg modules live in rtl/amba/axil4/.

2.1.2 Key Features

- ✓ **Dynamic Clock Gating:** Automatic clock disable during idle
- ✓ **Configurable Idle Threshold:** Programmable idle count before gating
- ✓ **Full Functional Equivalence:** Identical to base modules
- ✓ **Status Monitoring:** cg_gating and cg_idle status outputs
- ✓ **Runtime Bypass:** cfg_cg_enable = 0 disables gating entirely

There is **no scan/test-mode bypass port**. clock_gate_ctrl exposes only cfg_cg_enable; a DFT flow must drive that low (or bypass the ICG cell in the scan-insertion step) to hold the clock free-running during scan.

2.2 Additional Parameters (All _cg Modules)

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2 ^N -1 cycles)

All other parameters identical to base module.

2.3 Additional Ports (All _cg Modules)

2.3.1 Clock Gating Configuration

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0=disabled, 1=enabled)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating

2.3.2 Clock Gating Status

Port	Direction	Width	Description
cg_gating	Output	1	Clock currently gated (1=gated, 0=running)
cg_idle	Output	1	No activity was seen on the previous cycle (registered ~wakeup)

All other ports are identical to the base module, with one exception: the base module's busy output is **not** brought out on the _cg wrapper. It is consumed internally as one of the wakeup terms. Use cg_idle (or cg_gating) for interface idle detection on a _cg module.

There is no cumulative gated-cycle counter on these modules. If you need one, count cg_gating in the surrounding logic on the ungated clock.

2.4 Usage Example

```
axil4_master_rd_cg #(
    // Base parameters
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH(4) // Up to 15 idle cycles
) u_axil_rd_cg (
    .aclk(axi_clk),
    .aresetn(axi_resetn),

    // Clock gating configuration
    .cfg_cg_enable(1'b1), // Enable gating
    .cfg_cg_idle_count(4'd5), // Gate after 5 idle cycles

    // AXIL signals (identical to base module)
    .fub_araddr(cpu_araddr),
    // ... rest of AR/R signals ...

    // Clock gating status
    .cg_gating(rd_clk_gated),
    .cg_idle(rd_idle)
);
```

2.5 Clock Gating Behavior

The `_cg` wrapper builds a wakeup term from the module's channel activity and the base module's internal busy, and hands it to `amba_clock_gate_ctrl`, which registers it and drives `clock_gate_ctrl` and its ICG cell. For `axil4_master_rd_cg` the terms are:

```
assign user_valid = fub_arvalid || fub_rready || int_busy;
assign axi_valid  = m_axil_arvalid || m_axil_rvalid;
```

Gating Conditions (All Must Be True): 1. No activity in `user_valid` or `axi_valid` 2. Idle for `cfg_cg_idle_count` consecutive cycles (the idle counter counts down to 0 and holds) 3. Gating enabled (`cfg_cg_enable = 1`)

Ungating Conditions (Any Triggers Ungating): 1. Any `*valid` signal asserted on either side of the module 2. `fub_rready` asserted, or the base module still busy

Gating latency: `cfg_cg_idle_count + 1` clocks after the internal wakeup deasserts, which is `cfg_cg_idle_count + 2` clocks after the last bus activity on AXI4-Lite (one extra for the `r_wakeup` flop).

Ungating latency: activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. The `AXIL4_cg` wrappers drive `user_valid/axi_valid` combinational, so there is exactly **1 register stage** — `r_wakeup` inside `amba_clock_gate_ctrl` — and the first gated-clock rising edge available to the block arrives **2 clocks** after activity asserts, not 0. Note that `clock_gate_ctrl` contributes no flop of its own: `w_gate_enable` is combinational from wakeup straight into the ICG enable, so the second clock is the released edge itself rather than a further register stage. The wrapper covers this by forcing the module's ready outputs low while `cg_gating` is asserted, for example:

```
assign fub_arready  = cg_gating ? 1'b0 : int_arready;
assign m_axil_rready = cg_gating ? 1'b0 : int_rready;
```

so a request presented while gated is simply not accepted until the clock is running again. No data is lost, but a transaction that arrives into a gated interface pays the wakeup cost before its first handshake.

2.6 Configuration Guidelines

2.6.1 Idle Count Selection

Aggressive Gating (Register Access):

```
.cfg_cg_idle_count(4'd1)    // Gate after 1 idle cycle
// AXI4-Lite typical: Burst register access with idle gaps
```

Moderate Gating (Balanced):

```
.cfg_cg_idle_count(4'd5)    // Gate after 5 idle cycles
// General purpose configuration
```

Conservative Gating (Low Latency):

```
.cfg_cg_idle_count(4'd10)   // Gate after 10 idle cycles
// Critical control paths, minimal overhead
```

Disable Gating:

```
.cfg_cg_enable(1'b0)        // Clock gating disabled
// 100% utilization or ultra-low-latency paths
```

2.6.2 When to Use Clock Gating

✓ **Recommended For:** - Register access patterns (idle between accesses) - Low-duty-cycle control interfaces - Power-constrained systems

✗ **Not Recommended For:** - 100% utilization scenarios - Ultra-low-latency critical paths - Continuous register polling

2.7 Power Savings Estimates

2.7.1 Typical Savings (AXI4-Lite Register Access)

The percentages below are **rough expectations for dynamic clock power in the gated block only**, relative to the same module with `cfg_cg_enable = 0` at the same frequency. They are not measured power-analysis results, they exclude leakage (which gating does not reduce), and they exclude the clock tree upstream of the ICG cell. Run power analysis on your own target before relying on them.

Traffic Pattern	Duty Cycle	Idle Count	Est. Dynamic Power Savings
Burst register reads	20%	1	35-40%
Sporadic writes	10%	1	40-45%
Periodic polling	50%	5	20-25%
High activity	80%	5	5-10%

Note: AXI4-Lite often has better power savings than AXI4 due to: - Single-beat transactions (more idle gaps) - Simpler control (faster idle detection) - Register access patterns (bursty by nature)

2.8 Complete Documentation

For full clock gating details, see: - [AXI4 Clock Gating Guide](#) - Complete reference with: - Detailed gating behavior and state machine - Power savings calculations and examples - Dynamic configuration patterns - Synthesis considerations

AXIL4-specific notes: - Same architecture as AXI4 clock gating - Typically better power savings (bursty register access) - Simpler configuration (fewer outstanding transactions)

2.9 Related Documentation

2.9.1 Base Modules

- [axil4_master_rd](#) - Base read master
- [axil4_master_wr](#) - Base write master
- [axil4_slave_rd](#) - Base read slave
- [axil4_slave_wr](#) - Base write slave
- [Monitor modules](#) - Base monitor modules

2.9.2 Architecture

- [AXI4 Clock Gating Guide](#) - Complete reference
 - [RTLamba Overview](#) - Complete AMBA subsystem
 - [AXIL4 Index](#) - AXIL4 module index
-

Last Updated: 2026-07-19

2.10 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

3 AXIL4 Master Read

Module: axil4_master_rd.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

3.1 Overview

The AXIL4 Master Read module provides a buffered AXI4-Lite read interface for master devices. AXI4-Lite is a simplified subset of AXI4 designed for control register access with reduced signal count and single-beat transactions only.

3.1.1 Key Features

- ✓ **AXI4-Lite Protocol:** Simplified read-only interface (AR and R channels)
- ✓ **Single-Beat Transactions:** No burst support (always 1 transfer)
- ✓ **Buffered Channels:** Configurable skid buffers for timing closure
- ✓ **Elastic Buffering:** Decouples frontend and backend timing
- ✓ **Activity Monitoring:** Busy signal for clock gating
- ✓ **Minimal Latency:** 1-cycle buffer overhead per channel

3.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width (typically 32)
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AR	int	2	AR channel skid buffer depth, in entries
SKID_DEPTH_R	int	4	R channel skid buffer depth, in entries

3.2.1 Derived Parameters

Each channel's skid buffer stores the whole channel payload as one packed vector, so the derived *Size parameters are just the sum of the widths of the signals concatenated into that buffer.

Parameter	Calculation	Description
AW	AXIL_ADDR_WIDTH	Internal address width alias

Parameter	Calculation	Description
DW	AXIL_DATA_WIDTH	Internal data width alias
ARSize	AW+3	AR packet size: {araddr[AW-1:0], arprot[2:0]} — the 3 is ARPROT
RSize	DW+2	R packet size: {rdata[DW- 1:0], rresp[1:0]} — the 2 is RRESP

3.3 Port Groups

3.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	Active-low asynchronous reset

3.3.2 Frontend AXI4-Lite Read Interface (Input Side)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_araddr | Input | AW | Read address | | fub_arprot | Input | 3 | Protection
 attributes | | fub_arvalid | Input | 1 | Address valid | | fub_arready | Output | 1 | Address
 ready (from buffer) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_rdata | Output | DW | Read data | | fub_rresp | Output | 2 | Read response
 (OKAY, SLVERR, DECERR) | | fub_rvalid | Output | 1 | Data valid (from buffer) | | fub_rready
 | Input | 1 | Data ready |

3.3.3 Backend AXI4-Lite Master Interface (Output Side)

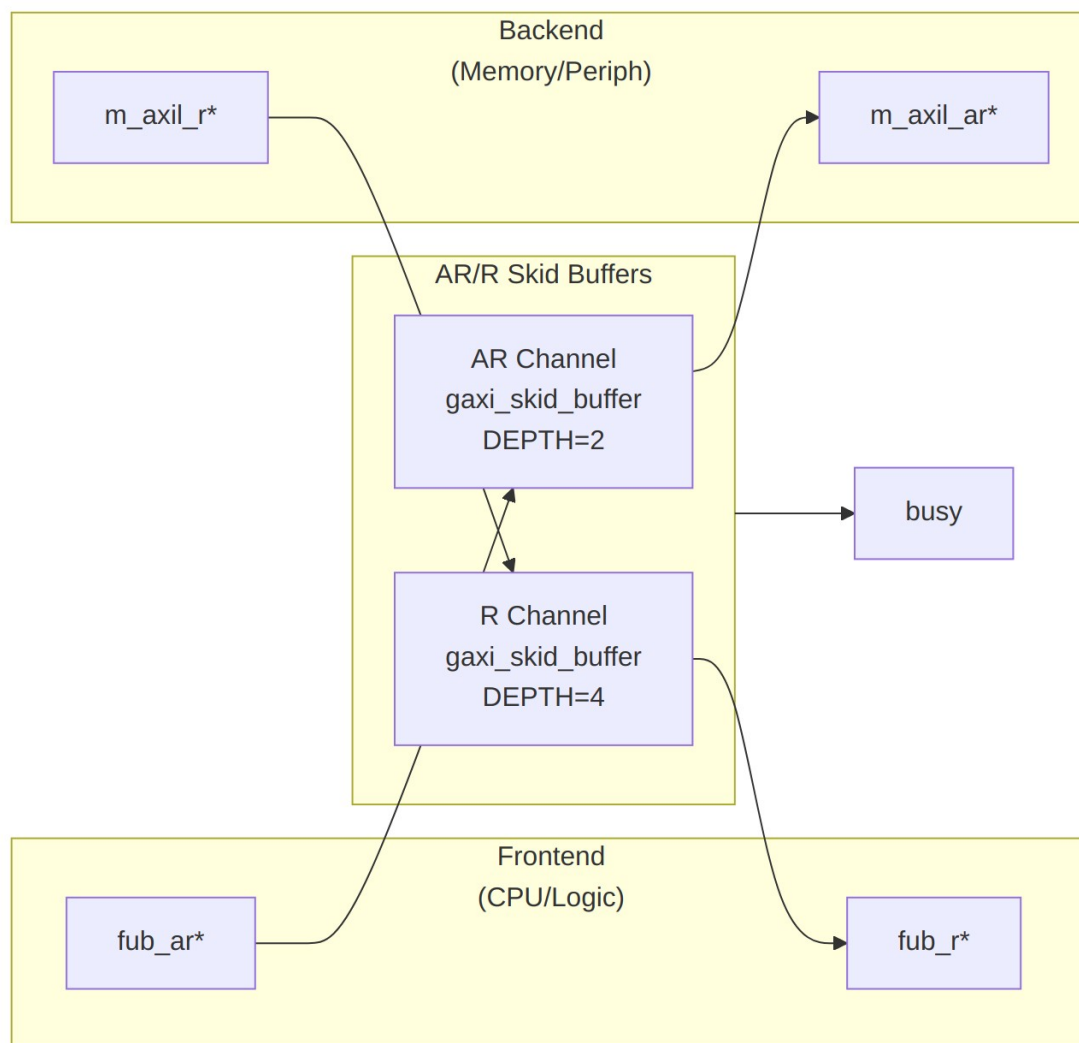
Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
 ————| | m_axil_araddr | Output | AW | Read address (to slave) | | m_axil_arprot |
 Output | 3 | Protection attributes | | m_axil_arvalid | Output | 1 | Address valid (from buffer)
 | | m_axil_arready | Input | 1 | Address ready (from slave) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | m_axil_rdata | Input | DW | Read data (from slave) | | m_axil_rresp | Input | 2

| Read response | | m_axil_rvalid | Input | 1 | Data valid (from slave) | | m_axil_rready |
Output | 1 | Data ready (to slave) |

3.3.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)



Module Architecture

3.4 Signal Flow

3.4.1 Read Transaction Sequence

Cycle 1: fub_araddr=0x1000, fub_arvalid=1, fub_arprot=0
fub_arready=1 (buffer accepts)

Cycle 2: AR buffer forwards to backend:
m_axil_araddr=0x1000, m_axil_arvalid=1
m_axil_arready=1 (slave accepts)

Cycle 3-5: Slave processes read

Cycle 6: m_axil_rdata=0xDEADBEEF, m_axil_rresp=OKAY
m_axil_rvalid=1, m_axil_rready=1 (R buffer accepts)

Cycle 7: R buffer forwards to frontend:
fub_rdata=0xDEADBEEF, fub_rresp=OKAY
fub_rvalid=1, fub_rready=1 (frontend accepts)

3.5 Usage Example

3.5.1 Basic Integration

```
axil4_master_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),      // 2 entries
    .SKID_DEPTH_R(4)       // 4 entries
) u_axil_master_rd (
    .aclk      (axi_clk),
    .aresetn   (axi_resetn),

    // Frontend interface (from CPU)
    .fub_araddr    (cpu_araddr),
    .fub_arprot    (cpu_arprot),
    .fub_arvalid   (cpu_arvalid),
    .fub_arready   (cpu_arready),

    .fub_rdata     (cpu_rdata),
    .fub_rresp     (cpu_rresp),
    .fub_rvalid    (cpu_rvalid),
    .fub_rready    (cpu_rready),
```

```

// Backend interface (to peripheral/memory)
.m_axil_araddr  (periph_araddr),
.m_axil_arprot  (periph_arprot),
.m_axil_arvalid (periph_arvalid),
.m_axil_arready (periph_arready),

.m_axil_rdata   (periph_rdata),
.m_axil_rresp   (periph_rresp),
.m_axil_rvalid  (periph_rvalid),
.m_axil_rready  (periph_rready),

// Status
.busy           (rd_busy)
);

```

3.5.2 Register Read Example (Testbench Stimulus)

The snippet below is **testbench stimulus, not RTL**. It uses an initial block and blocking assignments to drive the frontend ports and is not synthesizable. In a real design the frontend handshake is driven by an always_ff block or by the register-access state machine of the requesting logic.

```

// Read from control register at 0x1000
initial begin
    @(posedge aclk);
    fub_araddr  = 32'h1000;
    fub_arprot  = 3'b000; // Unprivileged, secure, data access
    fub_arvalid = 1'b1;

    @(posedge aclk);
    while (!fub_arready) @(posedge aclk); // Wait for buffer accept
    fub_arvalid = 1'b0;

    // Wait for read data
    fub_rready = 1'b1;
    @(posedge aclk);
    while (!fub_rvalid) @(posedge aclk);

    $display("Read data: 0x%08x, RRESP: %0d", fub_rdata, fub_rresp);
end

```

3.6 Design Notes

3.6.1 AXI4-Lite Protocol Constraints

Single-Beat Only: - No burst support (implicitly ARLEN=0, ARSIZE=log2(DW/8)) - Each AR transaction results in exactly 1 R transaction - No RLAST signal (always last)

No Out-of-Order: - No ARID/RID signals - Responses always in order - Simpler transaction tracking

Reduced Signals: - No ARCACHE, ARQOS, ARREGION, ARLOCK, ARSIZE, ARLEN, ARBURST - Only ARADDR and ARPROT for control

3.6.2 Buffer Depth Selection

SKID_DEPTH_AR and SKID_DEPTH_R are passed straight through to the DEPTH parameter of gaxi_skid_buffer. DEPTH is the **literal entry count**, not an exponent, and is constrained to one of {2, 4, 6, 8}.

AR Channel (Addresses): - **SKID_DEPTH_AR = 2** (2 entries): Typical for control registers - Increase if many back-to-back reads with slow slave response

R Channel (Data): - **SKID_DEPTH_R = 4** (4 entries): Accommodates pipeline latency - Increase for high-latency backends (DRAM, off-chip peripherals)

3.6.3 Busy Signal

busy is the OR of two persistent terms and two transient terms:

Term	Kind	Condition
AR buffer occupancy	Persistent	int_ar_count > 0
R buffer occupancy	Persistent	int_r_count > 0
Frontend presenting a new address	Transient	fub_arvalid
Backend presenting new data	Transient	m_axil_rvalid

The two transient terms can be high for a single cycle, so busy itself can pulse. This is intentional: the clock-gate controller does not gate on busy directly. amba_clock_gate_ctrl registers activity into a wakeup flop and then requires cfg_cg_idle_count consecutive idle cycles before gating, so a one-cycle handshake reloads the idle counter rather than causing the gate to oscillate. Consumers of busy outside the clock-gating path should qualify or stretch it themselves if they need a level signal.

Use for: - Clock gating control (see axil4_master_rd_cg) - Power management - Interface idle detection

3.6.4 PROT Encoding

ARPROT[2:0]	Privilege	Security	Type
3'b000	Unprivileged	Secure	Data
3'b001	Unprivileged	Secure	Instruction
3'b010	Unprivileged	Non-secure	Data
3'b011	Unprivileged	Non-secure	Instruction
3'b100	Privileged	Secure	Data
3'b101	Privileged	Secure	Instruction
3'b110	Privileged	Non-secure	Data
3'b111	Privileged	Non-secure	Instruction

Typical: 3'b000 (unprivileged secure data) for most peripherals

3.7 Performance Characteristics

3.7.1 Latency

Path	Cycles	Notes
Frontend → Backend (AR)	1	Skid buffer overhead
Backend → Frontend (R)	1	Skid buffer overhead
Total read latency	Slave latency + 2	AR + Slave + R

3.7.2 Throughput

Scenario	Rate	Notes
Back-to-back reads	1 read/cycle	If buffers not stalled
Typical peripheral	1 read/N cycles	N = slave response time

3.7.3 Resource Usage

These are **order-of-magnitude estimates**, not measured synthesis results — no target device, tool version, or optimization setting is attached to them. They scale with AXIL_DATA_WIDTH and the SKID_DEPTH_* settings. Synthesize for your own device before budgeting area.

Resource	Count	Notes
LUTs	~200	Estimate, 32-bit data, default depths
FFs	~150	Buffer state + control
BRAM	0	Skid buffers are flop-based

3.8 Related Modules

3.8.1 Core Modules

- [axil4_master_wr](#) - Write master counterpart
- [axil4_slave_rd](#) - Slave read interface
- [axil4_slave_wr](#) - Slave write interface

3.8.2 Monitor Modules

- [axil4_master_rd_mon](#) - Master read with monitoring (rtl/amba/monitor/)

3.8.3 Clock-Gated Variants

- [axil4_master_rd_cg](#) - Clock-gated version

3.8.4 Supporting Infrastructure

- [gaxi_skid_buffer](#) - Elastic buffering (rtl/amba/gaxi/)
-

3.9 References

3.9.1 Specifications

- ARM IHI 0022E: AMBA AXI and ACE Protocol Specification
- Chapter 4: AXI4-Lite (simplified subset)

3.9.2 Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem
- [AXI4 Master Read](#) - Full AXI4 comparison

3.9.3 Source Code

- RTL: rtl/amba/axil4/axil4_master_rd.sv

- Tests: `val/amba/test_axil4_master_rd.py`
 - Framework: `bin/TBClasses/components/axil4/`
-

Last Updated: 2026-07-19

3.10 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

4 AXIL4 Master Read Interface (Clock-Gated)

Module: `axil4_master_rd_cg.sv` **Base Module:** [axil4_master_rd](#) **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

4.1 Quick Reference

This is the **clock-gated variant** of [axil4_master_rd](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

- [AXIL4 Clock-Gated Variants Guide](#) (AXIL4-specific)
 - [Clock-Gated Variants Guide](#) (cross-protocol overview)
-

4.2 Summary

The `axil4_master_rd_cg` module adds power optimization to `axil4_master_rd` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
- ✓ **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the [AXIL4 Clock-Gated Variants Guide](#) for the estimate table and its caveats

- ✓ **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
 - ✓ **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module
-

4.3 Common Parameters

In addition to all `axil4_master_rd` parameters:

The `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
<code>CG_IDLE_COUNT_WIDTH</code>	int	4	Width of the idle counter; max idle count = $2^N - 1$

4.3.1 Additional Ports

Port	Direction	Width	Description
<code>cfg_cg_enable</code>	Input	1	Enable clock gating (0 = clock always running)
<code>cfg_cg_idle_count</code>	Input	<code>CG_IDLE_COUNT_WIDTH</code>	Idle cycles before gating
<code>cg_gating</code>	Output	1	Clock currently gated
<code>cg_idle</code>	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the `_cg` wrapper; it is consumed internally as a wakeup term. All other ports are identical to `axil4_master_rd`.

4.4 Quick Usage

```
axil4_master_rd_cg #(
    // Base module parameters (see axil4_master_rd.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
```

```

        .SKID_DEPTH_R(4),

        // Clock gating (see CG guide for details)
        .CG_IDLE_COUNT_WIDTH(4)
    ) u_cg (
        .aclk(clk),
        .aresetn(rst_n),

        // Clock gating control (runtime, not parameters)
        .cfg_cg_enable(1'b1),
        .cfg_cg_idle_count(4'd5),

        // ... all other ports same as axil4_master_rd, except `busy`,
        // which is replaced by cg_gating / cg_idle
        .cg_gating(clk_gated),
        .cg_idle(if_idle)
    );

```

4.5 Documentation

- **Base Module Functionality:** [axil4_master_rd.md](#)
 - **AXIL4 Clock Gating Guide:** [axil4_clock_gating_guide.md](#)
 - **Cross-Protocol Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

Last Updated: 2026-07-19

4.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

5 AXIL4 Master Write

Module: axil4_master_wr.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

5.1 Overview

The AXIL4 Master Write module provides a buffered AXI4-Lite write interface for master devices. AXI4-Lite is a simplified subset of AXI4 designed for control register access with reduced signal count and single-beat transactions only.

5.1.1 Key Features

- ✓ **AXI4-Lite Protocol:** Simplified write-only interface (AW, W, B channels)
 - ✓ **Single-Beat Transactions:** No burst support (always 1 transfer)
 - ✓ **Buffered Channels:** Configurable skid buffers for all 3 channels
 - ✓ **Byte Strobes:**WSTRB support for partial word writes
 - ✓ **Elastic Buffering:** Decouples frontend and backend timing
 - ✓ **Activity Monitoring:** Busy signal for clock gating
 - ✓ **Minimal Latency:** 1-cycle buffer overhead per channel
-

5.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width (typically 32)
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AW	int	2	AW channel skid buffer depth, in entries
SKID_DEPTH_W	int	2	W channel skid buffer depth, in entries

Parameter	Type	Default	Description
SKID_DEPTH_B	int	2	B channel skid buffer depth, in entries

5.2.1 Derived Parameters

Each channel's skid buffer stores the whole channel payload as one packed vector, so the derived *Size parameters are just the sum of the widths of the signals concatenated into that buffer.

Parameter	Calculation	Description
AW	AXIL_ADDR_WIDTH	Internal address width alias
DW	AXIL_DATA_WIDTH	Internal data width alias
AWSize	AW+3	AW packet: {awaddr[AW-1:0], awprot[2:0]} — the 3 is AWPROT
WSize	DW+(DW/8)	W packet: {wdata[DW-1:0], wstrb[DW/8-1:0]} — one strobe bit per byte lane
BSize	2	B packet: bresp[1:0] only

5.3 Port Groups

5.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	Active-low asynchronous reset

5.3.2 Frontend AXI4-Lite Write Interface (Input Side)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_awaddr | Input | AW | Write address | | fub_awprot | Input | 3 | Protection
 attributes | | fub_awvalid | Input | 1 | Address valid | | fub_awready | Output | 1 | Address
 ready (from buffer) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_wdata | Input | DW | Write data | | fub_wstrb | Input | DW/8 | Write strobes
 (byte lane enables) | | fub_wvalid | Input | 1 | Data valid | | fub_wready | Output | 1 | Data
 ready (from buffer) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_bresp | Output | 2 | Write response (OKAY, SLVERR, DECERR) | | fub_bvalid
 | Output | 1 | Response valid (from buffer) | | fub_bready | Input | 1 | Response ready |

5.3.3 Backend AXI4-Lite Master Interface (Output Side)

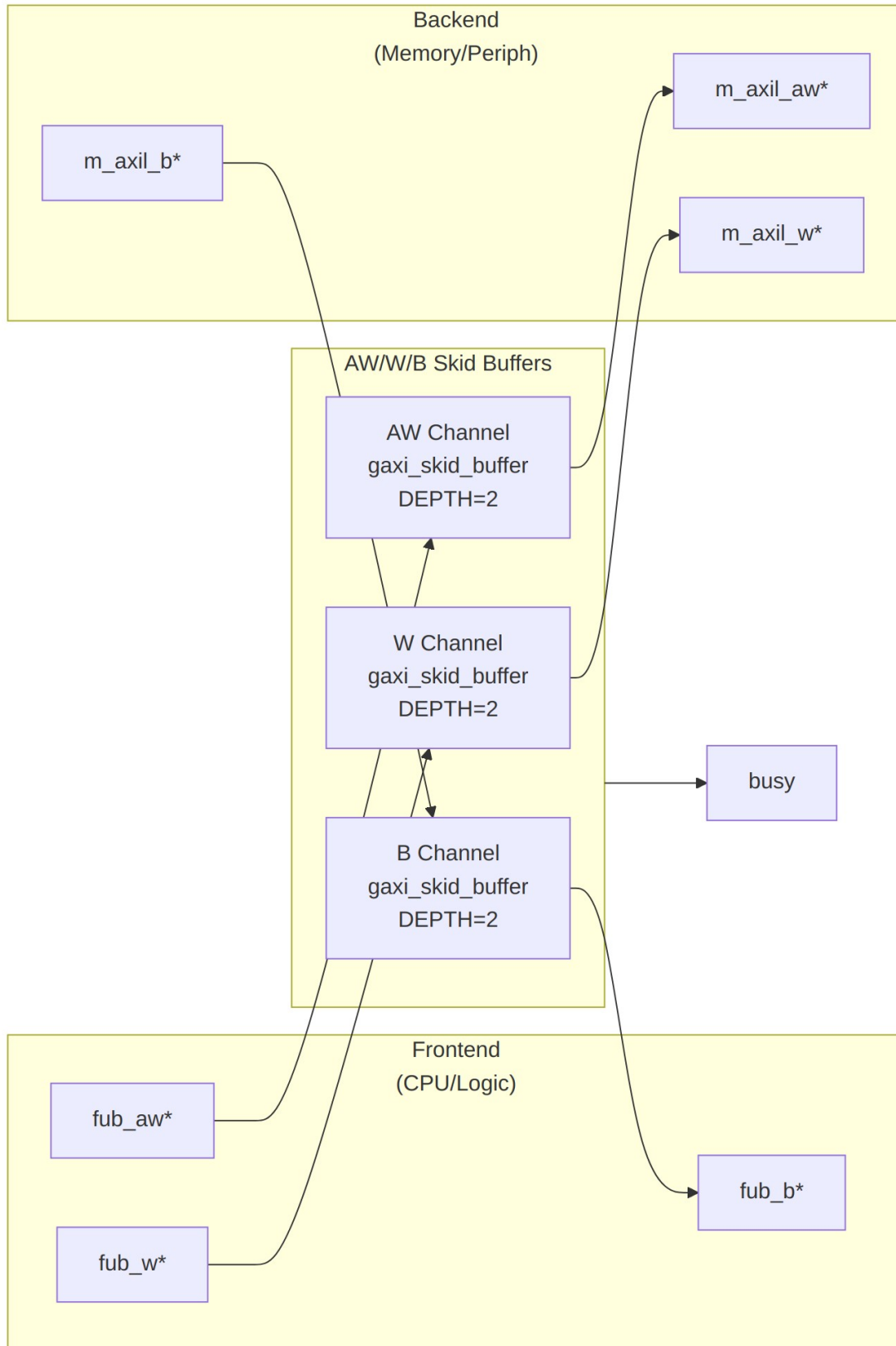
Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
 ————| | m_axil_awaddr | Output | AW | Write address (to slave) | | m_axil_awprot |
 Output | 3 | Protection attributes | | m_axil_awvalid | Output | 1 | Address valid (from buffer)
 | | m_axil_awready | Input | 1 | Address ready (from slave) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
 ————| | m_axil_wdata | Output | DW | Write data (to slave) | | m_axil_wstrb | Output |
 DW/8 | Write strobes | | m_axil_wvalid | Output | 1 | Data valid (from buffer) | |
 m_axil_wready | Input | 1 | Data ready (from slave) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
 ————| | m_axil_bresp | Input | 2 | Write response (from slave) | | m_axil_bvalid | Input
 | 1 | Response valid (from slave) | | m_axil_bready | Output | 1 | Response ready (to slave) |

5.3.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)



5.4 Signal Flow

5.4.1 Write Transaction Sequence

Cycle 1: fub_awaddr=0x2000, fub_awvalid=1, fub_awprot=0
fub_wdata=0xCAFEBAFE, fub_wstrb=4'b1111, fub_wvalid=1
fub_awready=1, fub_wready=1 (buffers accept)

Cycle 2: Buffers forward to backend:
m_axil_awaddr=0x2000, m_axil_awvalid=1
m_axil_wdata=0xCAFEBAFE, m_axil_wstrb=4'b1111,
m_axil_wvalid=1
m_axil_awready=1, m_axil_wready=1 (slave accepts)

Cycle 3-5: Slave processes write

Cycle 6: m_axil_bresp=OKAY, m_axil_bvalid=1
m_axil_bready=1 (B buffer accepts)

Cycle 7: B buffer forwards to frontend:
fub_bresp=OKAY, fub_bvalid=1
fub_bready=1 (frontend accepts)

5.5 Usage Example

5.5.1 Basic Integration

```
axil4_master_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),      // 2 entries
    .SKID_DEPTH_W(2),       // 2 entries
    .SKID_DEPTH_B(2)        // 2 entries
) u_axil_master_wr (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Frontend interface (from CPU)
    .fub_awaddr    (cpu_awaddr),
    .fub_awprot    (cpu_awprot),
    .fub_awvalid   (cpu_awvalid),
```



```

.fub_awready      (cpu_awready),

.fub_wdata        (cpu_wdata),
.fub_wstrb        (cpu_wstrb),
.fub_wvalid       (cpu_wvalid),
.fub_wready       (cpu_wready),

.fub_bresp        (cpu_bresp),
.fub_bvalid       (cpu_bvalid),
.fub_bready       (cpu_bready),

// Backend interface (to peripheral/memory)
.m_axil_awaddr    (periph_awaddr),
.m_axil_awprot    (periph_awprot),
.m_axil_awvalid   (periph_awvalid),
.m_axil_awready   (periph_awready),

.m_axil_wdata     (periph_wdata),
.m_axil_wstrb     (periph_wstrb),
.m_axil_wvalid    (periph_wvalid),
.m_axil_wready    (periph_wready),

.m_axil_bresp     (periph_bresp),
.m_axil_bvalid    (periph_bvalid),
.m_axil_bready    (periph_bready),

// Status
.busy             (wr_busy)
);

```

5.5.2 Register Write Example (Testbench Stimulus)

The two snippets below are **testbench stimulus, not RTL**. They use an `initial` block and blocking assignments to drive the frontend ports and are not synthesizable. In a real design the frontend handshake is driven by an `always_ff` block or by the register-access state machine of the requesting logic.

```

// Write to control register at 0x2000
initial begin
    @(posedge aclk);

    // Present address and data simultaneously
    fub_awaddr  = 32'h2000;
    fub_awprot  = 3'b000;
    fub_awvalid = 1'b1;

```

```

fub_wdata    = 32'hDEADBEEF;
fub_wstrb    = 4'b1111; // All bytes valid
fub_wvalid   = 1'b1;

@(posedge aclk);
while (!(fub_awready && fub_wready)) @(posedge aclk);
fub_awvalid  = 1'b0;
fub_wvalid   = 1'b0;

// Wait for write response
fub_bready   = 1'b1;
@(posedge aclk);
while (!fub_bvalid) @(posedge aclk);

$display("Write response: BRESP=%0d", fub_bresp);
end

```

5.5.3 Partial Write Example (Byte Strobes)

```

// Write only upper 16 bits to 32-bit register
fub_awaddr    = 32'h2004;
fub_awprot    = 3'b000;
fub_awvalid   = 1'b1;

fub_wdata     = 32'hABCD0000; // Data in upper half
fub_wstrb     = 4'b1100;      // Only upper 2 bytes valid
fub_wvalid    = 1'b1;

// Result: Register[31:16] = 0xABCD, Register[15:0] unchanged

```

5.6 Design Notes

5.6.1 AXI4-Lite Protocol Constraints

Single-Beat Only: - No burst support (implicitly AWLEN=0, AWSIZE=log2(DW/8)) - Each AW+W transaction results in exactly 1 B response - No WLAST signal (always last and only beat)

No Out-of-Order: - No AWID/BID signals - Responses always in order - AW and W can arrive in any order as far as **this module** is concerned: the AW and W skid buffers are fully independent and nothing here pairs an address with its data. The protocol permits either order, but a downstream slave may still require AW before W (or at least in the same cycle) to decode the target before it can consume the data. Check the slave's requirements — this module will not reorder for you.

Reduced Signals: - No AWCACHE, AWQOS, AWREGION, AWLOCK, AWSIZE, AWLEN, AWBURST - Only AWADDR and AWPROT for control

5.6.2 Buffer Depth Selection

SKID_DEPTH_AW, SKID_DEPTH_W, and SKID_DEPTH_B are passed straight through to the DEPTH parameter of gaxi_skid_buffer. DEPTH is the **literal entry count**, not an exponent, and is constrained to one of {2, 4, 6, 8}.

AW Channel (Addresses): - SKID_DEPTH_AW = 2 (2 entries): Typical for control registers - Increase if many back-to-back writes with slow slave

W Channel (Data): - SKID_DEPTH_W = 2 (2 entries): Matches AW depth - Should be >= SKID_DEPTH_AW to avoid W channel stalls

B Channel (Responses): - SKID_DEPTH_B = 2 (2 entries): Responses are single-beat - Smaller depth acceptable if frontend can always accept

5.6.3 Write Strobe (WSTRB) Usage

32-bit Data Width: |WSTRB[3:0] | Bytes Written | Use Case | | | | |
 4'b0001 | [7:0] | Byte write to offset 0 | | 4'b0010 | [15:8] | Byte write to offset 1 | | 4'b0011 | [15:0] | Half-word write | | 4'b1100 | [31:16] | Upper half-word | | 4'b1111 | [31:0] | Full word write |

64-bit Data Width: |WSTRB[7:0] | Bytes Written | Use Case | | | | |
 8'b0000_0001 | [7:0] | Byte write | | 8'b0000_1111 | [31:0] | Lower word | | 8'b1111_0000 | [63:32] | Upper word | | 8'b1111_1111 | [63:0] | Full double-word |

5.6.4 Busy Signal

busy is the OR of three persistent terms and two transient terms:

Term	Kind	Condition
AW buffer occupancy	Persistent	int_aw_count > 0
W buffer occupancy	Persistent	int_w_count > 0
B buffer occupancy	Persistent	int_b_count > 0
Frontend presenting a new transaction	Transient	fub_awvalid fub_wvalid
Backend presenting a	Transient	m_axil_bvalid

Term	Kind	Condition
response		
The two transient terms can be high for a single cycle, so busy itself can pulse. This is intentional: the clock-gate controller does not gate on busy directly. <code>amba_clock_gate_ctrl</code> registers activity into a wakeup flop and then requires <code>cfg_cg_idle_count</code> consecutive idle cycles before gating, so a one-cycle handshake reloads the idle counter rather than causing the gate to oscillate. Use for: - Clock gating control (see <code>axil4_master_wr_cg</code>) - Power management - Interface idle detection		

5.7 Performance Characteristics

5.7.1 Latency

Path	Cycles	Notes
Frontend → Backend (AW)	1	Skid buffer overhead
Frontend → Backend (W)	1	Skid buffer overhead
Backend → Frontend (B)	1	Skid buffer overhead
Total write latency	Slave latency + 3	AW + W + Slave + B, best case

The +3 best case assumes the slave asserts `AWREADY` and `WREADY` in the **same** cycle, so the AW and W buffer traversals overlap. If the slave accepts the address and the data in different cycles, the write completes off the later of the two handshakes, and the total is that skew plus the slave and B latency. Backpressure on B adds further cycles.

5.7.2 Throughput

Scenario	Rate	Notes
Back-to-back writes	1 write/cycle	If buffers not stalled
Typical peripheral	1 write/N cycles	N = slave response time

5.7.3 Resource Usage

These are **order-of-magnitude estimates**, not measured synthesis results — no target device, tool version, or optimization setting is attached to them. They scale with AXIL_DATA_WIDTH and the SKID_DEPTH_* settings. Synthesize for your own device before budgeting area.

Resource	Count	Notes
LUTs	~300	Estimate, 32-bit data, 3 channels, default depths
FFs	~250	Buffer state + control
BRAM	0	Skid buffers are flop-based

5.8 Related Modules

5.8.1 Core Modules

- [axil4_master_rd](#) - Read master counterpart
- [axil4_slave_rd](#) - Slave read interface
- [axil4_slave_wr](#) - Slave write interface

5.8.2 Monitor Modules

- [axil4_master_wr_mon](#) - Master write with monitoring (rtl/amba/monitor/)

5.8.3 Clock-Gated Variants

- [axil4_master_wr_cg](#) - Clock-gated version

5.8.4 Supporting Infrastructure

- [gaxi_skid_buffer](#) - Elastic buffering (rtl/amba/gaxi/)
-

5.9 References

5.9.1 Specifications

- ARM IHI 0022E: AMBA AXI and ACE Protocol Specification
- Chapter 4: AXI4-Lite (simplified subset)

5.9.2 Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem
- [AXI4 Master Write](#) - Full AXI4 comparison

5.9.3 Source Code

- RTL: rtl/amba/axil4/axil4_master_wr.sv
 - Tests: val/amba/test_axil4_master_wr.py
 - Framework: bin/TBClasses/components/axil4/
-

Last Updated: 2026-07-19

5.10 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

6 AXIL4 Master Write Interface (Clock-Gated)

Module: axil4_master_wr_cg.sv **Base Module:** [axil4_master_wr](#) **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

6.1 Quick Reference

This is the **clock-gated variant** of [axil4_master_wr](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [AXIL4 Clock-Gated Variants Guide](#) (AXIL4-specific)

→ [Clock-Gated Variants Guide](#) (cross-protocol overview)

6.2 Summary

The `axil4_master_wr_cg` module adds power optimization to `axil4_master_wr` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
- ✓ **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the [AXIL4 Clock-Gated Variants Guide](#) for the estimate table and its caveats
- ✓ **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
- ✓ **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module

6.3 Common Parameters

In addition to all [axil4_master_wr](#) parameters:

The `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
<code>CG_IDLE_COUNT_WIDTH</code>	int	4	Width of the idle counter; max idle count = $2^N - 1$

6.3.1 Additional Ports

Port	Direction	Width	Description
<code>cfg_cg_enable</code>	Input	1	Enable clock gating (0 = clock always running)
<code>cfg_cg_idle_count</code>	Input	<code>CG_IDLE_COUNT_WIDTH</code>	Idle cycles before gating
<code>cg_gating</code>	Output	1	Clock currently gated
<code>cg_idle</code>	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the `_cg` wrapper; it is consumed internally as a wakeup term. All other ports are identical to `axil4_master_wr`.

6.4 Quick Usage

```
axil4_master_wr_cg #(
    // Base module parameters (see axil4_master_wr.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_master_wr, except `busy`,
    // which is replaced by cg_gating / cg_idle
    .cg_gating(clk_gated),
    .cg_idle(if_idle)
);
```

6.5 Documentation

- **Base Module Functionality:** [axil4_master_wr.md](#)
 - **AXIL4 Clock Gating Guide:** [axil4_clock_gating_guide.md](#)
 - **Cross-Protocol Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

Last Updated: 2026-07-19

6.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

7 AXIL4 Slave Read

Module: axil4_slave_rd.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

7.1 Overview

The AXIL4 Slave Read module provides a buffered AXI4-Lite read interface for slave devices (memory, peripherals). It accepts read requests from an interconnect/master and forwards them to backend logic with elastic buffering for timing closure.

7.1.1 Key Features

- ✓ **AXI4-Lite Slave:** Buffered read-only interface (AR and R channels)
- ✓ **Single-Beat Transactions:** No burst support
- ✓ **Elastic Buffering:** Decouples interconnect and backend timing
- ✓ **Activity Monitoring:** Busy signal for clock gating
- ✓ **Minimal Latency:** 1-cycle buffer overhead per channel

7.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AR	int	2	AR channel skid buffer depth, in entries
SKID_DEPTH_R	int	4	R channel skid buffer depth,

Parameter	Type	Default	Description
			in entries

7.3 Port Groups

7.3.1 Slave Interface (Input Side from Interconnect)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_araddr | Input | AW | Read address (from interconnect) | |
 s_axil_arprot | Input | 3 | Protection attributes | | s_axil_arvalid | Input | 1 | Address
 valid | | s_axil_arready | Output | 1 | Address ready (buffer status) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_rdata | Output | DW | Read data (to interconnect) | | s_axil_rresp |
 Output | 2 | Read response | | s_axil_rvalid | Output | 1 | Data valid | | s_axil_rready |
 Input | 1 | Data ready |

7.3.2 Backend Interface (Output Side to Memory/Logic)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_araddr | Output | AW | Read address (to backend) | | fub_arprot | Output | 3
 | Protection attributes | | fub_arvalid | Output | 1 | Address valid | | fub_arready | Input | 1
 | Address ready (from backend) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_rdata | Input | DW | Read data (from backend) | | fub_rresp | Input | 2 |
 Read response | | fub_rvalid | Input | 1 | Data valid | | fub_rready | Output | 1 | Data ready
 |

7.3.3 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)

7.4 Usage Example

```
axil4_slave_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4)
```

```

) u_axil_slave_rd (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_araddr (interconnect_araddr),
    .s_axil_arprot (interconnect_arprot),
    .s_axil_arvalid (interconnect_arvalid),
    .s_axil_arready (interconnect_arready),

    .s_axil_rdata   (interconnect_rdata),
    .s_axil_rresp   (interconnect_rresp),
    .s_axil_rvalid  (interconnect_rvalid),
    .s_axil_rready  (interconnect_rready),

    // Backend interface (to memory/peripheral)
    .fub_araddr     (mem_araddr),
    .fub_arprot     (mem_arprot),
    .fub_arvalid    (mem_arvalid),
    .fub_arready    (mem_arready),

    .fub_rdata      (mem_rdata),
    .fub_rresp      (mem_rresp),
    .fub_rvalid     (mem_rvalid),
    .fub_rready     (mem_rready),

    // Status
    .busy           (rd_busy)
);

```

7.5 Design Notes

- **Signal Flow:** Interconnect → AR Buffer → Backend → R Buffer → Interconnect
 - **Use Case:** Memory controllers, peripheral slaves, register blocks
 - **Buffering:** Allows backend to respond at its own pace without stalling interconnect
-

7.6 Related Modules

- [axil4_slave_wr](#) - Slave write counterpart

- [axil4_master_rd](#) - Master read interface
 - [axil4_slave_rd_mon](#) - Slave read with monitoring (rtl/amba/monitor/)
 - [axil4_slave_rd_cg](#) - Clock-gated version
-

Last Updated: 2026-07-19

7.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

8 AXIL4 Slave Read Interface (Clock-Gated)

Module: `axil4_slave_rd_cg.sv` **Base Module:** [axil4_slave_rd](#) **Location:** `rtl/amba/axil4/`
Status: ✓ Production Ready

8.1 Quick Reference

This is the **clock-gated variant** of [axil4_slave_rd](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

- [AXIL4 Clock-Gated Variants Guide](#) (AXIL4-specific)
 - [Clock-Gated Variants Guide](#) (cross-protocol overview)
-

8.2 Summary

The `axil4_slave_rd_cg` module adds power optimization to `axil4_slave_rd` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module

- ✓ **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the [AXIL4 Clock-Gated Variants Guide](#) for the estimate table and its caveats
 - ✓ **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
 - ✓ **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module
-

8.3 Common Parameters

In addition to all [axil4_slave_rd](#) parameters:

The `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
<code>CG_IDLE_COUNT_WIDTH</code>	int	4	Width of the idle counter; max idle count = $2^N - 1$

8.3.1 Additional Ports

Port	Direction	Width	Description
<code>cfg_cg_enable</code>	Input	1	Enable clock gating (0 = clock always running)
<code>cfg_cg_idle_count</code>	Input	<code>CG_IDLE_COUNT_WIDTH</code>	Idle cycles before gating
<code>cg_gating</code>	Output	1	Clock currently gated
<code>cg_idle</code>	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the `_cg` wrapper; it is consumed internally as a wakeup term. All other ports are identical to `axil4_slave_rd`.

8.4 Quick Usage

```
axil4_slave_rd_cg #(
    // Base module parameters (see axil4_slave_rd.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_slave_rd, except `busy`,
    // which is replaced by cg_gating / cg_idle
    .cg_gating(clk_gated),
    .cg_idle(if_idle)
);
```

8.5 Documentation

- **Base Module Functionality:** [axil4_slave_rd.md](#)
 - **AXIL4 Clock Gating Guide:** [axil4_clock_gating_guide.md](#)
 - **Cross-Protocol Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

Last Updated: 2026-07-19

8.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

9 AXIL4 Slave Write

Module: axil4_slave_wr.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

9.1 Overview

The AXIL4 Slave Write module provides a buffered AXI4-Lite write interface for slave devices (memory, peripherals). It accepts write requests from an interconnect/master and forwards them to backend logic with elastic buffering for timing closure.

9.1.1 Key Features

- ✓ **AXI4-Lite Slave:** Buffered write-only interface (AW, W, B channels)
- ✓ **Single-Beat Transactions:** No burst support
- ✓ **Byte Strobes:**WSTRB support for partial writes
- ✓ **Elastic Buffering:** Decouples interconnect and backend timing
- ✓ **Activity Monitoring:** Busy signal for clock gating
- ✓ **Minimal Latency:** 1-cycle buffer overhead per channel

9.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AW	int	2	AW channel skid buffer depth, in entries
SKID_DEPTH_W	int	2	W channel

Parameter	Type	Default	Description
			skid buffer depth, in entries
SKID_DEPTH_B	int	2	B channel skid buffer depth, in entries

9.3 Port Groups

9.3.1 Slave Interface (Input Side from Interconnect)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_awaddr | Input | AW | Write address (from interconnect) | |
 s_axil_awprot | Input | 3 | Protection attributes | | s_axil_awvalid | Input | 1 | Address valid | | s_axil_awready | Output | 1 | Address ready (buffer status) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_wdata | Input | DW | Write data (from interconnect) | | s_axil_wstrb | Input | DW/8 | Write strobes | | s_axil_wvalid | Input | 1 | Data valid | | s_axil_wready | Output | 1 | Data ready (buffer status) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_bresp | Output | 2 | Write response (to interconnect) | | s_axil_bvalid | Output | 1 | Response valid | | s_axil_bready | Input | 1 | Response ready |

9.3.2 Backend Interface (Output Side to Memory/Logic)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_awaddr | Output | AW | Write address (to backend) | | fub_awprot | Output | 3 | Protection attributes | | fub_awvalid | Output | 1 | Address valid | | fub_awready | Input | 1 | Address ready (from backend) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_wdata | Output | DW | Write data (to backend) | | fub_wstrb | Output | DW/8 | Write strobes | | fub_wvalid | Output | 1 | Data valid | | fub_wready | Input | 1 | Data ready (from backend) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_bresp | Input | 2 | Write response (from backend) | | fub_bvalid | Input | 1 | Response valid | | fub_bready | Output | 1 | Response ready |

9.3.3 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)

9.4 Usage Example

```
axil4_slave_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_axil_slave_wr (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_awaddr (interconnect_awaddr),
    .s_axil_awprot (interconnect_awprot),
    .s_axil_awvalid (interconnect_awvalid),
    .s_axil_awready (interconnect_awready),

    .s_axil_wdata   (interconnect_wdata),
    .s_axil_wstrb   (interconnect_wstrb),
    .s_axil_wvalid  (interconnect_wvalid),
    .s_axil_wready  (interconnect_wready),

    .s_axil_bresp   (interconnect_bresp),
    .s_axil_bvalid  (interconnect_bvalid),
    .s_axil_bready  (interconnect_bready),

    // Backend interface (to memory/peripheral)
    .fub_awaddr     (mem_awaddr),
    .fub_awprot     (mem_awprot),
    .fub_awvalid    (mem_awvalid),
    .fub_awready    (mem_awready),

    .fub_wdata      (mem_wdata),
    .fub_wstrb      (mem_wstrb),
    .fub_wvalid     (mem_wvalid),
    .fub_wready     (mem_wready),
```

```

.fub_bresp      (mem_bresp),
.fub_bvalid     (mem_bvalid),
.fub_bready     (mem_bready),

// Status
.busy           (wr_busy)
);

```

9.5 Design Notes

- **Signal Flow:** Interconnect → AW/W Buffers → Backend → B Buffer → Interconnect
 - **Use Case:** Memory controllers, peripheral slaves, register blocks
 - **Buffering:** Allows backend to respond at its own pace without stalling interconnect
 - **Write Strobes:** Backend must honor WSTRB for partial word writes
-

9.6 Related Modules

- [axil4_slave_rd](#) - Slave read counterpart
 - [axil4_master_wr](#) - Master write interface
 - [axil4_slave_wr_mon](#) - Slave write with monitoring (rtl/amba/monitor/)
 - [axil4_slave_wr_cg](#) - Clock-gated version
-

Last Updated: 2026-07-19

9.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

10 AXIL4 Slave Write Interface (Clock-Gated)

Module: `axil4_slave_wr_cg.sv` **Base Module:** [axil4_slave_wr](#) **Location:** `rtl/amba/axil4/`
Status: ✓ Production Ready

10.1 Quick Reference

This is the **clock-gated variant** of [axil4_slave_wr](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

- [AXIL4 Clock-Gated Variants Guide](#) (AXIL4-specific)
 - [Clock-Gated Variants Guide](#) (cross-protocol overview)
-

10.2 Summary

The `axil4_slave_wr_cg` module adds power optimization to `axil4_slave_wr` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** dynamic clock power in the gated block scales with idle fraction; see the [AXIL4 Clock-Gated Variants Guide](#) for the estimate table and its caveats
 - ✓ **Configurable:** idle threshold and enable, both at **runtime** via `cfg_cg_idle_count` / `cfg_cg_enable`
 - ✓ **Zero Overhead When Disabled:** `cfg_cg_enable = 0` holds the clock free-running, making behavior identical to the base module
-

10.3 Common Parameters

In addition to all [axil4_slave_wr](#) parameters:

The `_cg` wrapper adds exactly **one** parameter. Gating is enabled and tuned at runtime through ports, not through parameters — there is no `ENABLE_CLOCK_GATING` parameter, no `CG_IDLE_CYCLES` parameter, and no per-domain `CG_GATE_*` parameters on this module.

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle counter; max idle count = $2^N - 1$

10.3.1 Additional Ports

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0 = clock always running)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating
cg_gating	Output	1	Clock currently gated
cg_idle	Output	1	No activity on the previous cycle

The base module's busy output is **not** exposed on the _cg wrapper; it is consumed internally as a wakeup term. All other ports are identical to axil4_slave_wr.

10.4 Quick Usage

```
axil4_slave_wr_cg #(
    // Base module parameters (see axil4_slave_wr.md)
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2),

    // Clock gating (see CG guide for details)
    .CG_IDLE_COUNT_WIDTH(4)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),

    // Clock gating control (runtime, not parameters)
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(4'd5),

    // ... all other ports same as axil4_slave_wr, except `busy`,
    // which is replaced by cg_gating / cg_idle
```

```
.cg_gating(clk_gated),  
.cg_idle(if_idle)  
);
```

10.5 Documentation

- **Base Module Functionality:** [axil4_slave_wr.md](#)
 - **AXIL4 Clock Gating Guide:** [axil4_clock_gating_guide.md](#)
 - **Cross-Protocol Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

Last Updated: 2026-07-19

10.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)